VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

AI Text Translator Using Python and Flask

A9513 – Python Programming Laboratory

Submitted by
KERELLY NIHALINI REDDY
25881A05A1
BHUVANAGIRI VINAY
25881A05D0
Batch: B-21

Course Facilitator

Ms P.S. Madhavi
Assistant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

Academic Year – 2026-27

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A**++ Grade
Kacharam, Shamshabad, Hyderabad- 501 218,
Telangana, India.

Vibrant Vardhaman
25 Years of

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"AI Text Translator Using Python and Flask"** is carried out by **Kerelly Nihalini Reddy** and **Bhuvanagiri Vinay** with Roll Numbers **25881A05A1** and **25881A05D0** towards **A9513 - Python Programming Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2026-27.

Signature of the Course Faculty

Ms P.S.Madhavi
Assistant Professor,
Department of
Computer Science and
Engineering

Signature of the HOD
Dr. Y. Vijayalata
Professor, Head of the
Department & Dean
(Student Mentoring),
Department of Computer
Science and Engineering

Contents

1 Introduction....3

2 Literature Review....3

3 Objectives....4

4 Methodology....5

4.1 Architectural Diagram....5

4.2 Hardware and Software Requirements....5

4.3 Working Principle....6

4.4 Software Implementation....7

5 Results and Discussion....9

6 Mapping POs,PSOs and SDGs....11

6.1 Program Outcomes (POs) Mapping....11

6.2 Sustainable Development Goals (SDGs) Mapping....12

6.3 Program Specific Outcomes (PSOs) Mapping....12

7 Conclusion....12

Bibliography....13

Abstract

The **AI Text Translator** is a lightweight web application developed using Python and the Flask micro-framework that allows users to translate text between multiple languages through a simple browser interface. The user types or pastes text into a form, selects a target language from a drop-down list and submits the request. The Flask back end validates the input, builds a structured prompt and sends it to Google's Gemini generative AI model through the google-genai Python SDK. The translated text returned by the model is rendered back on the same page using a Jinja2 template, so no additional software is required on the client side.

The application supports nine target languages – Telugu, Hindi, Tamil, Kannada, Malayalam, English, French, German and Spanish – covering both Indian regional languages and widely used international languages. The prompt sent to the model instructs it to return only the translated text while preserving the meaning, names, numbers and punctuation, which keeps the output clean and ready to use. Basic error handling reports empty input and API failures to the user. Testing with sample sentences produced accurate and natural translations in Telugu, Hindi, Tamil, German and English, showing that a large language model combined with a minimal Flask front end can deliver a practical and extensible translation tool with very little code.

Keywords: Machine Translation, Artificial Intelligence, Generative AI, Gemini API, Python, Flask, Natural Language Processing, Jinja2, Web Application, Multilingual.

1 Introduction

Language is the primary medium through which people communicate, learn and conduct business, yet the world's linguistic diversity often becomes a barrier to information access. India alone recognises 22 scheduled languages and hundreds of dialects, while much of the content on the internet, in education and in software is available mainly in English. As a result, students, professionals and ordinary citizens frequently need to translate text between their regional language and other languages in order to understand documents, messages, study material or official notices.

Machine translation has evolved considerably over the past decades. Early systems relied on hand-written grammar rules and bilingual dictionaries, which produced stiff, word-for-word output. Statistical approaches learned phrase mappings from large parallel corpora, and neural machine translation later introduced encoder–decoder and Transformer architectures that model whole sentences and context. Today, large language models (LLMs) trained on multilingual data can translate between many languages simply by being given a clear instruction, without any task-specific training or maintenance of translation dictionaries.

This project, **AI Text Translator Using Python and Flask**, takes advantage of that capability. A small Flask web application accepts text and a target language from the user, constructs a carefully worded prompt and forwards it to the Gemini model using the google-genai SDK. The model's reply is shown to the user on the same web page. The prompt rules tell the model to translate the complete text, preserve names, numbers and punctuation, and return nothing but the translation, which makes the output directly usable.

2 Literature Review

Research in machine translation has progressed through several generations. Rule-based and dictionary-based systems translate by looking up words in bilingual lexicons and applying hand-written reordering rules. They are transparent and inexpensive for a small vocabulary, but they cannot handle idioms, ambiguity or the complex morphology of languages such as Telugu, Tamil and Malayalam, and the dictionaries must be maintained manually. Statistical machine translation, notably phrase-based models (Koehn et al., 2003), improved fluency by learning from parallel corpora, but it required large amounts of aligned data and struggled with long-range dependencies.

Neural machine translation changed the field. Bahdanau et al. (2015) introduced an attention mechanism that lets a model align source and target words while translating, and Vaswani et al. (2017) proposed the Transformer, which replaced recurrence with self-attention and became the foundation of modern language models. Brown et al. (2020) showed that very large pre-trained models can perform translation in a few-shot or even zero-shot manner when the task is described in the prompt, and the Gemini family of multimodal models (Gemini Team, 2023) extends this ability across many languages and tasks.

Despite this progress, many student-level translation tools are still dictionary-based and translate word by word, while general-purpose online translators offer little scope for customisation or learning. Prompt-based translation with a hosted LLM addresses these gaps: the instructions (for example, "preserve names and numbers") can be changed simply by editing the prompt, support for a new language only requires adding an entry to the drop-down list, and full sentences are translated in context. Limitations remain, including

dependence on an internet connection and API access, variable quality for low-resource languages and the possibility of occasional errors, so the output should be reviewed for critical use.

3 Objectives

- **To design and develop a web-based translation application using Python and Flask.**

The first objective is to build a functional web application in which a Flask server handles GET and POST requests, reads the text and target language submitted through an HTML form, and returns the translated result on the same page. The project aims to keep the code modular and readable, separating server logic (app.py), presentation (index.html) and styling (style.css).

- **To integrate a generative AI model for accurate, context-aware translation.**

The application uses the google-genai Python SDK to call the Gemini model. A structured prompt is built for every request, instructing the model to translate the complete text, preserve the meaning, names, numbers and punctuation, and to return only the translated text without explanations. This objective ensures natural, sentence-level translation instead of word-for-word substitution.

- **To support multiple Indian and international languages through a simple interface.**

The interface offers a text area, a language drop-down and a single Translate button. Nine target languages are available: Telugu, Hindi, Tamil, Kannada, Malayalam, English, French, German and Spanish. The design is deliberately minimal so that users with no technical background can use it immediately.

- **To provide input validation and error handling, and to test the system with sample inputs.**

Empty input is rejected with a friendly message, and exceptions raised during the API call are caught and displayed instead of crashing the application. The system is then tested with different sentences and target languages to verify the correctness and consistency of the output.

4 Methodology

4.1 Architectural Diagram

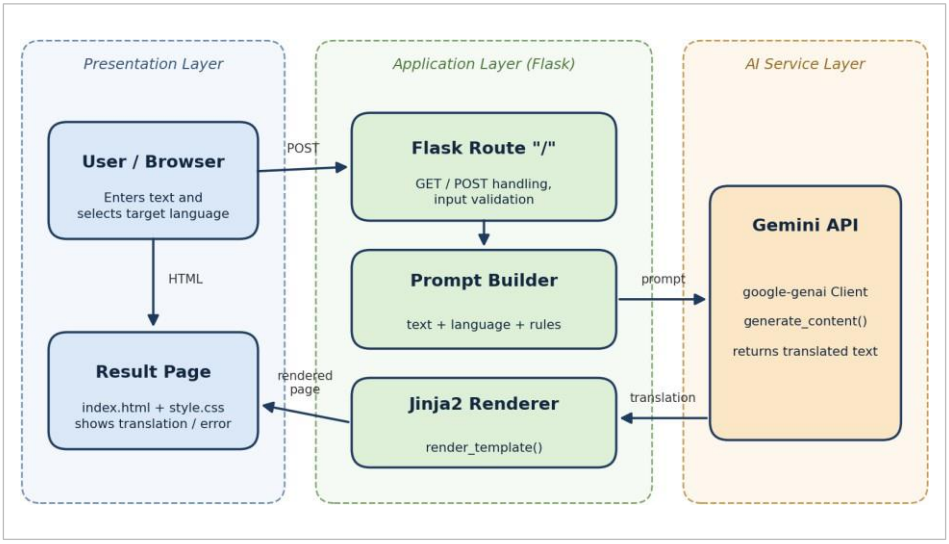


Figure 1: Architectural Diagram

The system follows a simple three-layer, client–server architecture. The **Presentation Layer** consists of the web browser and the HTML/CSS template. It displays the text area, the language drop-down and the Translate button, and later shows either the translated text or an error message. Because the form is submitted with the POST method, the page is simply re-rendered with the result.

The **Application Layer** is the Flask server. A single route ("/") serves both GET requests (showing the empty form) and POST requests (processing a translation). It reads the submitted text and target language, validates that the text is not empty, builds the prompt and finally renders the template through Jinja2, passing the translation or the error message to the page.

The **AI Service Layer** is the Gemini model, accessed through the google-genai client. The prompt is sent with the "generate_content" call and the model returns the translated text, which the server trims and forwards to the template. Keeping the AI call behind one function-level block makes it easy to change the model or even replace the provider without touching the user interface.

4.2 Hardware and Software Requirements

Hardware:

- Intel/AMD Processor
- Minimum 4 GB RAM
- Standard Display and Input Devices
- Active internet connection (required to reach the Gemini API)

Software:

- Python 3.13 (64-bit)
- Flask web framework (with Jinja2 templating)

- google-genai Python SDK (Google Gen AI SDK)
- Gemini API key, provided through the environment (read automatically by genai.Client())
- HTML5 and CSS3 for the user interface
- Visual Studio Code as the IDE
- Any modern web browser (Google Chrome used for testing)

4.3 Working Principle

The working principle of the AI Text Translator is based on sending a well-structured instruction, together with the user's text, to a generative language model and displaying the model's answer. The process consists of the following steps:

- **Input:** The user opens the application in a browser (the server listens on port 5000 on all network interfaces), types or pastes the text into the text area and chooses a target language from the drop-down list.
- **Request:** On clicking Translate, the browser sends the form data (text and target_language) to the Flask route using HTTP POST.
- **Validation:** The server strips leading and trailing spaces. If the text is empty, the error message "Please enter some text." is returned.
- **Prompt construction:** The server builds a prompt that names the target language and lists the rules: return only the translated text, do not explain, preserve meaning, names, numbers and punctuation, do not add information, and translate the complete text.
- **AI call:** The prompt is passed to the Gemini model with client.models.generate_content(). The model generates the translation.
- **Response handling:** The returned text is stripped and stored in the translation variable. If any exception occurs (for example a network or API error), it is caught and shown as "Translation error".
- **Output:** The index.html template is rendered with the translation or error, and the browser displays it below the form.

The drop-down list contains the following options. Each option's value is the language code that is placed in the prompt, and the model interprets it as the target language.

Table 1: Languages supported by the application

Code	Language	Code	Language
te	Telugu	en	English
hi	Hindi	fr	French
ta	Tamil	de	German
kn	Kannada	es	Spanish
ml	Malayalam		

Example problem: The following sample inputs were given to the application during testing.

Table 2: Sample translation test cases

#	Input text	Target	Output produced	Result
1	How can i help you	Telugu	నేను మీకు ఎలా సహాయపడగలను	Correct
2	How can i help you	Hindi	मैं आपकी ककस प्रकार सहायता कर सकता हूँ	Correct
3	How can i help you	Tamil	உங்களுக்கு நான் எவ்வாறு உதவ முடியும்?	Correct
4	the courage to keep moving forward even when things are difficult.	Hindi	जब चीजें मुश्किल ह ोें तब भी आगे बढ़ते रहने का साहस।	Correct
5	Python is a simple and powerful programming language used to build applications, automate tasks, and analyze data.	German	Python ist eine einfache und leistungsstarke Programmiersprache, die zum Erstellen von Anwendungen, Automatisieren von Aufgaben und Analysieren von Daten verwendet wird.	Correct
6	आज मौसम बहुत अच्छा है और मैं अपने द स् ोें केसाथ बाहर घूमने जा रहा हूँ।	English	The weather is very nice today and I am going out with my friends.	Correct

4.4 Software Implementation

The project is organised in the standard Flask layout, shown in Figure 2: a static folder holding style.css, a templates folder holding index.html, and the application file app.py at the root. The screenshot also shows the Flask development server log in the VS Code terminal, where each form submission appears as a POST request answered with status 200.

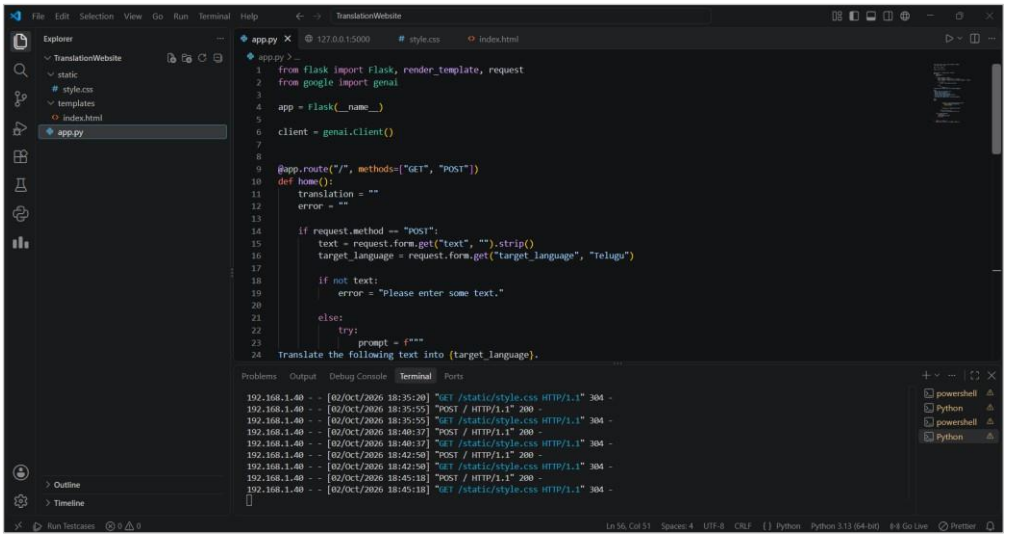


Figure 2: Project structure in VS Code and Flask server log

app.py

```
from flask import Flask, render_template, request
from google import genai

app = Flask(__name__)

client = genai.Client()
```

```

@app.route("/", methods=["GET", "POST"])
def home():
    translation = ""
    error = ""

    if request.method == "POST":
        text = request.form.get("text", "").strip()
        target_language = request.form.get("target_language", "Telugu")

        if not text:
            error = "Please enter some text."

        else:
            try:
                prompt = f"""
Translate the following text into {target_language}.

Rules:
- Return only the translated text.
- Do not explain anything.
- Preserve the original meaning.
- Preserve names, numbers and punctuation.
- Do not add extra information.
- Translate the complete text, including sentences.

Text:
{text}
"""

                response = client.models.generate_content(
                    model="gemini-3.5-flash-lite",
                    contents=prompt
                )

                translation = response.text.strip()

            except Exception as e:
                error = f"Translation error: {e}"

    return render_template(
        "index.html",
        translation=translation,
        error=error
    )


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

app.py creates the Flask application and the Gemini client, defines the single route that handles both GET and POST, validates the input, builds the prompt, calls generate_content() and passes the result or error to the template.

templates/index.html

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta  name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Translation Application</title>

    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>

<body>

    <div class="container">

        <h1>Translation Application</h1>

        <p>Translate text into different languages</p>

        <form method="POST">

            <textarea
                name="text"
                placeholder="Enter any text here..."
                required
            ></textarea>

            <br>

            <label for="target_language">Translate to:</label>

            <select name="target_language" id="target_language">

                <option value="te">Telugu</option>
                <option value="hi">Hindi</option>
                <option value="ta">Tamil</option>
                <option value="kn">Kannada</option>
                <option value="ml">Malayalam</option>
                <option value="en">English</option>
                <option value="fr">French</option>
```

```
                    <option value="de">German</option>
                    <option value="es">Spanish</option>

                </select>

                <br>

                <button type="submit">Translate</button>

            </form>

            {% if translation %}

            <div class="result">

                <h2>Translation</h2>

                <p>{{ translation }}</p>

            </div>

            {% endif %}

            {% if error %}

            <div class="error">

                <p>{{ error }}</p>

            </div>

            {% endif %}

        </div>

    </body>

    </html>
```

index.html contains the form (text area, language drop-down and submit button) and uses Jinja2 conditions ({% if translation %} and {% if error %}) to display the result or the error message only when they exist.

static/style.css

```
    body {
        font-family: Arial, sans-serif;
        background-color: #f2f2f2;
        text-align: center;
        margin: 0;
        padding: 0;
    }

    .container {
        width: 600px;
        margin: 100px auto;
        background-color: white;
        padding: 30px;
        border-radius: 10px;
    }

    h1 {
        font-size: 35px;
    }

    textarea {
        width: 90%;
        height: 120px;
        padding: 10px;
        font-size: 18px;
    }

    button {
        margin-top: 20px;
        padding: 12px 30px;
        font-size: 18px;
        cursor: pointer;
    }

    .result {
        margin-top: 30px;
    }
```

style.css centres the content in a white card on a light-grey background and sets simple, readable font sizes for the heading, text area and button.

5 Results and Discussion

The application was run on the Flask development server and opened from a browser on the local network. The following figures show the user interface and the outputs obtained for

different inputs and target languages.

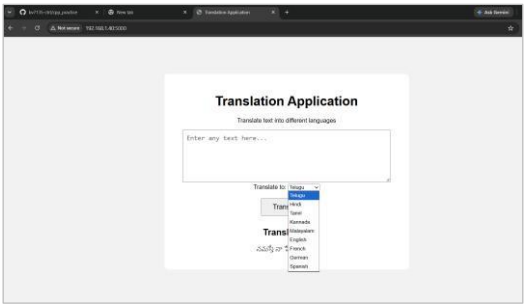


Figure 3: User interface with language drop-down

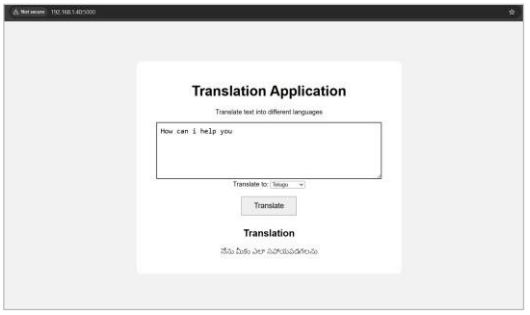


Figure 4: English to Telugu

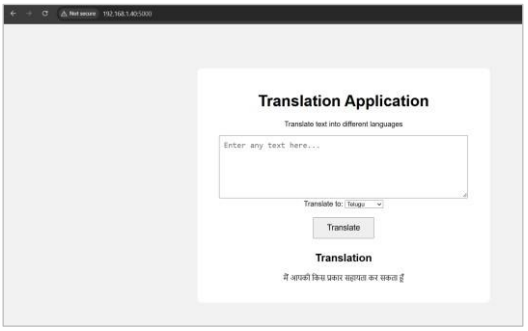


Figure 5: English to Hindi

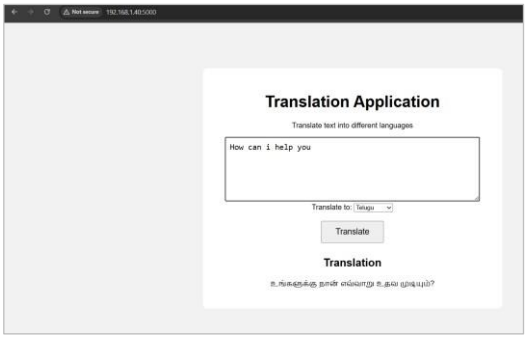


Figure 6: English to Tamil

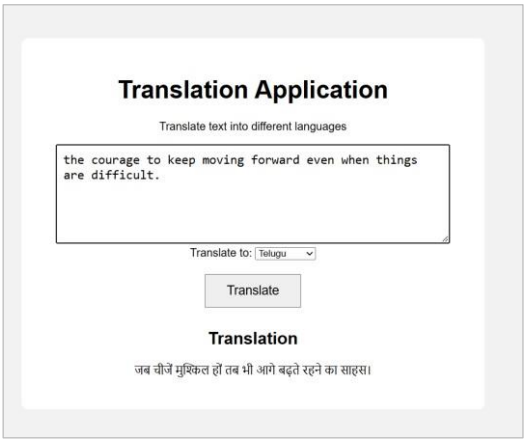


Figure 7: English sentence to Hindi

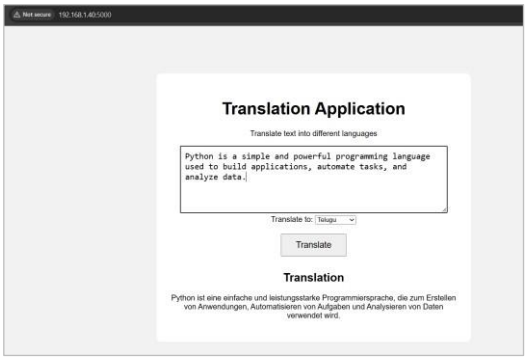


Figure 8: English to German

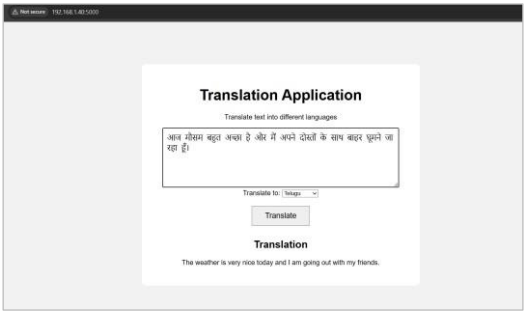


Figure 9: Hindi to English

Summary of Results

The translator returned correct and fluent output for every test case: English to Telugu, Hindi, Tamil and German, and Hindi to English. Both short phrases and longer sentences

were translated completely, with punctuation such as the question mark in Tamil and the danda in Hindi handled appropriately, and the technical term "Python" was preserved in the German output. Only the translated text was returned, without explanations, as required by the prompt rules.

Interpretation

The results show that prompt-based translation with a hosted LLM gives natural, context-aware output without maintaining any dictionary or training data. The same code handles Indian scripts (Telugu, Hindi, Tamil) and Latin-script languages, and the source language does not have to be specified because the model identifies it automatically. Each translation needs an internet connection and a valid API key, and the form is reset after every submission, so the language must be re-selected.

Achievement of Objectives

The project met its objectives. A Flask web application was built with a clean separation of server logic, template and styling; the Gemini model was integrated through the google-genai SDK; nine languages are selectable from a simple interface; and input validation and exception handling keep the application stable. Testing with the sample inputs confirmed the correctness of the output.

6 Mapping POs, PSOs and SDGs

6.1 Program Outcomes (POs) Mapping

Table 3: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied programming, web-development and natural language processing concepts to build a working translation application using Python and Flask.
PO2	Problem Analysis	Analysed the language-barrier problem and chose a prompt-based LLM approach instead of a dictionary-based method to translate full sentences accurately.
PO3	Design/Development of Solutions	Designed a three-layer solution (HTML interface, Flask server, Gemini API) with input validation, prompt design and error handling.
PO4	Conduct Investigations of Complex Problems	Tested the system with different sentences and target languages and examined the correctness, completeness and consistency of the outputs.
PO5	Engineering Tool Usage	Used Python, Flask, Jinja2, the google-genai SDK, HTML/CSS, VS Code and a web browser to develop, run and test the application.
PO8	Individual and Collaborative Team work	Divided coding, testing and documentation work between the two team members and collaborated on reviews and improvements.
PO9	Communication	Presented the design, source code, outputs and findings clearly through this report and the working web interface.

PO No.	Program Outcome	Relevance
PO11	Life-Long Learning	Learned the Flask framework and generative-AI APIs through self-study and hands-on experimentation, which supports continued learning of new technologies.

6.2 Sustainable Development Goals (SDGs) Mapping

Table 4: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 4	Quality Education	Helps learners understand study material and information in their own language, making education more inclusive and accessible.
SDG 8	Decent Work and Economic Growth	Builds practical skills in web development and AI integration that improve employability in IT and technology-driven industries.
SDG 9	Industry, Innovation and Infrastructure	Demonstrates innovative use of generative AI services in a lightweight web application and promotes software innovation.
SDG 10	Reduced Inequalities	Reduces the language gap by making content accessible in Indian regional languages as well as international languages.

6.3 Program Specific Outcomes (PSOs) Mapping

Table 5: Mapping of Program Specific Outcomes (PSOs) to Project Relevance

PSO No.	Description
PSO1	Apply programming, artificial intelligence, and web development concepts to design and implement practical software applications using Python and modern frameworks.
PSO2	Develop solutions to real-world problems by integrating programming tools, APIs, and AI technologies with appropriate software engineering practices.
PS03	Analyze, test, and improve software applications using suitable development tools and technologies to deliver reliable and user-friendly solutions.

7 Conclusion

This project successfully implemented an AI Text Translator using Python and Flask. By combining a minimal Flask web application with the Gemini generative model, the system lets users translate text into Telugu, Hindi, Tamil, Kannada, Malayalam, English, French, German and Spanish through a simple web page. The carefully worded prompt ensures that only the translated text is returned while meaning, names, numbers and punctuation are preserved. Testing showed accurate and natural translations, demonstrating that modern AI services can replace large hand-built dictionaries and rule sets with very little code.

For future enhancements, the application can retain the entered text and the selected language after submission, detect the source language automatically and display it, and add speech input and text-to-speech output. Further improvements include document and file translation, a translation history, a copy-to-clipboard button, asynchronous requests (AJAX), response caching, secure management of the API key and deployment on a cloud platform. A

quantitative evaluation of translation quality using metrics such as BLEU would further strengthen the system.

Bibliography

Online Resources

[1] Flask Documentation – https://flask.palletsprojects.com/

[2] Google Gen AI SDK for Python – https://googleapis.github.io/python-genai/

[3] Google AI for Developers, Gemini API Documentation – https://ai.google.dev/gemini-api/docs

[4] Jinja Template Designer Documentation – https://jinja.palletsprojects.com/

Research article references

[1] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. Advances in NeuralInformation Processing Systems (NeurIPS), 30.

[2] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. International Conference on Learning Representations (ICLR).

[3] Koehn, P., Och, F. J., & Marcu, D. (2003). Statistical Phrase-Based Translation. Proceedings of HLT-NAACL.

[4] Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language Models are Few-Shot Learners. Advances in Neural Information Processing Systems (NeurIPS), 33.

[5] Gemini Team, Google. (2023). Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.1180